

PHast-R: Bit-Parallel Perfect Hashing with Patterns and Cuckoo Repair

Sebastiano Vigna

Dipartimento di Informatica, Università degli Studi di Milano

Abstract

PHast and its fast-to-build variant PHast⁺ are minimal perfect hash functions with the fastest known queries: a key is mapped to a bucket by a linear function of its hash, and a fixed-width per-bucket seed selects the positions of the keys of the bucket inside small overlapping slices of the output range. PHast⁺ uses additive placement, so that feasible seeds can be found with bit-parallel operations, but it needs about 0.2 more bits per key than PHast. We analyze where this space goes: with a minimal output range, every bumped key leaves a hole that must be encoded, and the rigidity of additive placement both forces more bumping and causes *self-collisions* that no seed can resolve. We then propose PHast-R, in which the seed selects one of R independent offset patterns in addition to a shift, and in which construction *repairs* failing buckets by cuckoo-style eviction, detecting with a bit-parallel half adder the shifts at which exactly one key is blocked. Queries perform the same operations as in PHast⁺, plus two shifts and a multiplication that does not depend on the access to the seed. With 8-bit seeds, PHast-R matches the space of PHast (1.92 bits per key) while building 4.4 times faster; with 10-bit seeds it reaches 1.85 bits per key, the space of PHast, 12 times faster, and improves on all PHast⁺ variants at the same construction time. Finally, we study the limits of repair: we prove that when keys can move only within windows of W slots no placement of n keys on n slots, however found, can avoid bumping about $\sqrt{\pi n/2} - W$ keys, and we show that most failures of the sweep cannot be repaired at any depth, which explains why shallow repairs are sufficient.

1 Introduction

A *minimal perfect hash function* (MPHF) maps a static set of n keys bijectively to $[0..n)$; it requires at least $\lg e \approx 1.443$ bits per key (we use $\lg$ for the binary logarithm and $\ln$ for the natural logarithm). Among the many practical constructions, *bucket placement* gives the fastest queries: keys are hashed to buckets, and a per-bucket seed chooses positions for the keys of the bucket so that no two keys collide.

PHast [1] pushes this approach to its limit. A key with hash c goes to bucket $\lfloor cB/2^{64} \rfloor$; the seed s of the bucket, stored in a plain array of S -bit values, places the key at $\lfloor c(m-L+1)/2^{64} \rfloor + p(s, c)$, that is, inside a *slice* of length L whose start grows linearly with c . Buckets for which no seed works are *bumped* (seed 0) and handled by further levels; an Elias–Fano sequence [3, 4] maps their outputs to the free slots of the first level. A query thus reads a single seed for almost all keys. Seeds are assigned by a sweep over the buckets that keeps a small window of buckets in a priority queue, favoring large buckets, and chooses for each bucket the feasible seed that minimizes the sum of the positions of its keys.

In PHast, $p(s, c)$ is a pseudorandom function, so finding the best seed means trying all $2^S - 1$ of them; construction takes hundreds of nanoseconds per key. PHast⁺ uses instead the additive placement $p(s, c) = (c \bmod L) + s - 1$: the feasibility of 64 consecutive seeds of a bucket is the complement of the bitwise OR of 64-bit windows of the occupancy bitmap, one per key, and the best seed is the first feasible one. Construction becomes an order of magnitude faster, but with

8-bit seeds space grows from 1.92 to 2.12 bits per key; *wrapping* variants partially close the gap at the cost of slower construction.

This paper makes four contributions:

- an analysis of the space of PHast⁺ showing that its excess comes from bumping, and that a large part of it is caused by self-collisions inherent to additive placement (Section 2);
- PHast-R, a structure in which the seed chooses one of R independent offset patterns and a shift, and whose construction repairs failing buckets by evicting the owner of the only blocking slot, detected by a bit-parallel half adder (Section 3);
- an implementation in the Rust crate `sux` and an experimental comparison with the reference implementation of PHast and PHast⁺ (Section 4);
- a lower bound on the number of bumped keys that holds for every placement in which keys move within windows of fixed length, and an analysis of the limits of repair, showing that most failures of the sweep cannot be repaired at any depth, and that repair makes the sweep fail more often (Section 5).

2 The Space of PHast⁺

All measurements in this section refer to 10^7 random keys, 8-bit seeds, expected bucket size $\lambda = 5.25$ and slice length $L = 512$, the space-optimal configuration of PHast⁺. They have been obtained on the reference implementation of PHast⁺ (the `ph` crate, using GxHash as hash function), whose first level we inspect through read-only accessors.

Holes are bumped keys. The first level of PHast⁺ has output range $m = n$. Every key is either placed in a distinct slot or bumped, so the number of free slots (*holes*) is exactly the number of bumped keys. Each hole costs about $2 + \lg(1/\beta)$ bits in the Elias–Fano remapping, where β is the fraction of bumped keys, and each bumped key must be placed again by the following levels. With S -bit seeds the space is thus approximately

$$\frac{S}{\lambda(1 - \beta)} + \beta \left(2 + \lg \frac{1}{\beta} \right),$$

and since the entropy of a random set of βn holes is already about $\beta n(\lg(1/\beta) + \lg e)$, the encoding of holes cannot be substantially improved: *the only lever is β* . In our configuration $\beta = 7.6\%$, and bumping costs 0.59 of the 2.12 bits per key (0.47 for Elias–Fano, 0.13 for the further levels). We will show in Section 5 that on large key sets no placement, however found, can avoid bumping altogether.

Additive placement is rigid. Additive placement translates a fixed pattern: two keys of a bucket whose base positions coincide collide for every seed. For a bucket of size k this happens with probability about $k(k - 1)/(2L)$, so a fraction $\lambda^2/(2L) \approx 2.7\%$ of the buckets cannot be placed at all (we measure 2.6%); they are 38.5% of the bumped buckets and contain about $(\lambda^2 + 2\lambda)/(2L) \approx 3.7\%$ of the keys (we measure 3.6%). Moreover, consecutive shifts are strongly correlated trials. Seeds, on the other hand, are almost incompressible: their empirical entropy is 7.71 bits, because the shift mostly encodes how far the leftmost key of the bucket lies behind the full part of its slice.

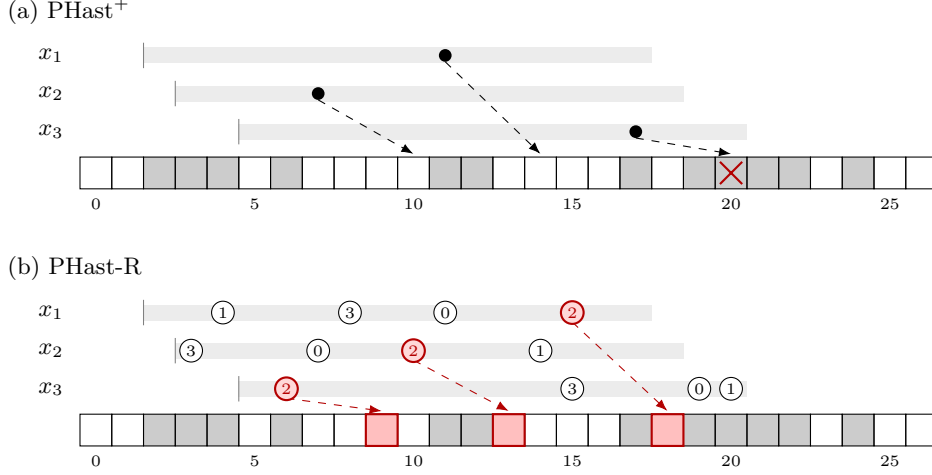


Figure 1: Placing a bucket with three keys x_1 , x_2 , x_3 using 4-bit seeds; gray bars are the slices of the keys ($L = 16$), and shaded slots are occupied. (a) In PHast⁺ each key has a single base position (dot), and the seed s translates all bases by $s - 1$: the 15 seeds translate the same rigid pattern, and in this example all of them hit an occupied slot (the dashed arrows show $s = 4$), so the bucket is bumped. (b) In PHast-R each key has $R = 4$ independent base positions, one per pattern (circles), and the seed selects a pattern r and a shift $d \in [0 \dots 4]$; here the only feasible choice is $r = 2$ and $d = 3$, that is, $s = dR + r = 14$.

3 PHast-R

Placement. Each key has a 64-bit hash h , from which we derive a second 64-bit value o (see below). Let $R = 2^\rho$ be the number of *patterns*, $L = 2^\ell$ the slice length with $\ell \leq 64/R$, and $D = 2^S/R$ the number of shifts. The bucket of a key is $\text{hi}(hB)$ and the start of its slice is $\sigma(h) = \text{hi}(h(m - L - D + 2))$, where $\text{hi}(x)$ denotes the 64 most significant bits of a 128-bit product. The r -th pattern assigns to the key the in-slice offset

$$\omega_r(o) = \lfloor o/2^{64r/R} \rfloor \bmod L,$$

that is, the ℓ bits of o starting at bit $64r/R$: different patterns are independent. A seed $s \in [1 \dots 2^S]$ encodes the pair $r = s \bmod R$, $d = \lfloor s/R \rfloor$ (the pair $r = d = 0$ is not available), and the key is mapped to

$$\sigma(h) + \omega_r(o) + d.$$

In other words, whereas in PHast⁺ all seeds translate the same rigid pattern of base positions, in PHast-R the seed first chooses among R independent patterns, and then translates the chosen one (Figure 1). As in PHast, seed 0 means that the bucket is bumped. A query computes the same quantities as in PHast⁺, plus two shifts and the multiplication deriving o (see below), which is independent of the access to the seed. The encoding makes it unnecessary to extract r : since $(s \cdot 64/R) \bmod 64 = r \cdot 64/R$, $\omega_r(o)$ is obtained by shifting o right by $s \cdot 64/R$ modulo 64 and masking, whereas d is s shifted right by ρ bits. We set $o = h \cdot c \bmod 2^{64}$ for an odd constant c . Keys of the same bucket share the upper $\approx \lg B$ bits of h , and their slices start at about the same position; but bit j of o depends on all bits $0, \dots, j$ of h , so every offset depends on the lower bits of h , which vary independently among the keys of a bucket. (Since ω_{R-1} uses bits of o up to 63, taking $o = h$ would make the last pattern nearly constant within a bucket.) With $R = 1$ and $o = h$, PHast-R is essentially PHast⁺.

Search. For each pattern, the best shift is found as in PHast⁺: the occupancy words starting at the base positions $\sigma(h_i) + \omega_r(o_i)$ of the k keys are Ored, and the first zero bit is the smallest

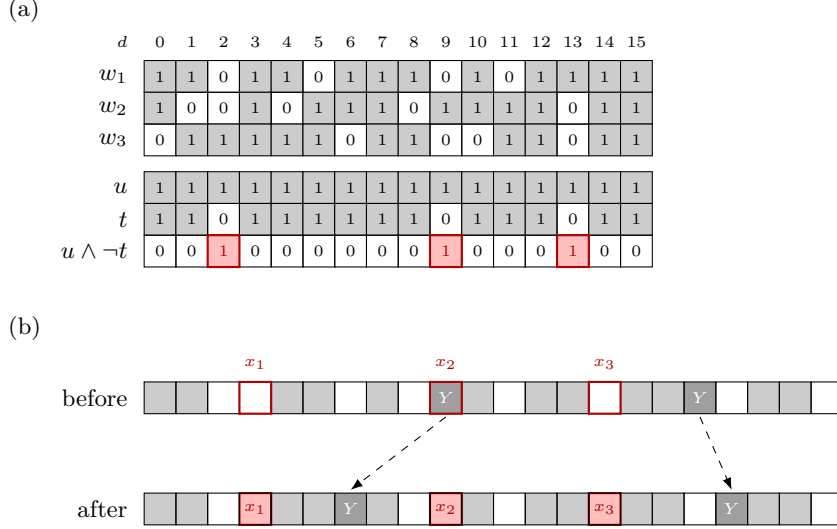


Figure 2: Search and repair. (a) The occupancy words w_1, w_2, w_3 read at the base positions of the three keys of a bucket for a given pattern (16 shifts shown instead of 64): bit d of w_i tells whether the slot at shift d from the base position of the i -th key is occupied. The first zero of $u = w_1 \vee w_2 \vee w_3$ would be the smallest feasible shift, but here there is none. The half adder accumulates in t the shifts at which at least two keys are blocked, so $u \wedge \neg t$ marks the shifts at which exactly one key is blocked: $d = 2$ (by x_3), $d = 9$ (by x_2), and $d = 13$ (by x_1). (b) Repair with $d = 9$: the owner array shows that the slot blocking x_2 belongs to a bucket Y with two keys; Y is evicted, the bucket is placed, and Y is placed again with a new seed. Had Y found no place, the eviction would have been undone, and the next candidate would have been tried.

feasible shift (Figure 2(a)); for $S = 8$ and $R = 4$ we have $D = 64$, so one word per key suffices. Among patterns, we keep the one minimizing the sum of positions, skipping a pattern as soon as the sum of its bases exceeds the best sum found. Since all R patterns of a bucket collide only with probability about $(k(k-1)/2L)^R$, self-collisions disappear for $R \geq 2$, and independent patterns decorrelate the trials.

Repair. When no pattern admits a feasible shift, PHast-R tries to make room. Using the same words $w_1, \dots, w_k$, the half adder

$$t \leftarrow t \vee (u \wedge w_i), \quad u \leftarrow u \vee w_i$$

computes in $u \wedge \neg t$ the shifts at which *exactly one* key is blocked. For each such candidate, in order of increasing sum of positions, an *owner* array parallel to the cyclic occupancy bitmap gives the bucket Y occupying the blocking slot. If Y is still entirely inside the active part of the window, we evict Y , place the current bucket, and place Y again (Figure 2(b)); if Y fails, it may in turn evict another bucket (with a quarter of the candidates), up to a maximum depth. If Y cannot be placed, the eviction is undone and the next candidate (we keep one per distinct Y) is tried. A failing bucket is bumped only after 16 unsuccessful candidates. Repair turns the greedy sweep of PHast into a local search, as in PtrHash [5], but it is triggered only by failures, and bumping remains as a safety valve: the structure stays minimal and needs no slack in the output range.

Slice length. As in PHast, the slice length depends on the size of the level: the first and last $L + D$ positions are reached by fewer slices than the others, so small levels need shorter slices. We use $L = 64$ when the range of the level is smaller than 2500, 128 below 4000, 256 below 70 000, 512 below 600 000, and a maximum, which is a parameter, above (and always $L \leq m/2$); the

thresholds have been tuned experimentally for $S = 8$. With repair, longer slices than in PHast⁺ pay off for large levels: for $S = 8$ the best maximum is $L = 1024$, whereas PHast⁺ uses $L = 512$. PHast uses a similar table with different thresholds (64 below 1300, 128 below 9500, 256 below 12 000, 512 below 140 000, then 1024 for $S < 12$), whereas PHast⁺ uses a fixed length depending only on S , limited by the range of the level. The priority weights used by the sweep depend on the slice length, and are chosen for each level accordingly; those for $S = 8$ and $L = 512$ or $L = 1024$, and for $S = 10$ and $L = 2048$, have been retuned for PHast-R by coordinate descent (the others are those of PHast⁺).

Levels. As in PHast⁺, keys of bumped buckets are hashed again at each following level with a different seed, and the new hash h' determines their bucket and slice. Offsets are provided by $o \oplus h'$, where o is the value of the first level: a query reaching further levels has already computed o (in parallel with the access to the seed of the first level), and needs a further hash of the key at each level, but no further multiplication. Since h' is independent of h , the offsets of a key at each level are independent of those of the previous levels and of the new bucket and slice. Levels after the first use the same scheme with range equal to their number of keys. As in PHast⁺, an Elias–Fano sequence maps the outputs of the levels after the first to the holes of the first level.

Colliding signatures. Keys with the same 64-bit hash h at the first level have the same slice and the same offsets, so their bucket self-collides for every seed and it is bumped; at the following levels they are hashed again, and thus separated. As in PHast⁺, 64-bit hashes are therefore sufficient for any number of keys, and no larger signature is necessary. Construction would fail only if two keys had the same hash at all levels, which happens in practice only for duplicate keys: to report duplicates before building the levels, keys whose hashes at the first level coincide are hashed again with a different seed, and keys whose hashes coincide again are considered duplicates.

Last level. When at most 4096 keys are left, a last level uses the same placement scheme with a range larger by 25% plus 16 slots, expected bucket size $\min(\lambda, 3)$, and no bumping, hashing the keys again with a different seed (and, every eight attempts, enlarging the range) in the unlikely case of failure. PHast⁺ behaves differently: when at most 8192 keys are left (4096 for the variants with wrapping), it builds a level of regular PHast (pseudorandom placement, all 256 seeds of 8 bits tried for each bucket, expected bucket size 4, no bumping) with range $1.2(k + 10)$, where k is the number of keys, rehashing all keys with a new seed in case of failure; queries reaching this level hash the key again. In particular, for small sets (up to about 10^5 keys, as PHast⁺ bumps 7.6% of the keys) all bumped keys go directly from the first level to a level of regular PHast.

Parallel construction. As in PHast, buckets are split into chunks separated by gaps of

$$\left\lfloor \frac{(L + D - 1)B}{m - L - D + 2} \right\rfloor + 1$$

buckets: the slice of bucket b starts at about $b(m - L - D + 2)/B$, and the positions of a key lie at most $L + D - 2$ slots after the start of its slice, so (considering the rounding of slice starts) the positions of different chunks cannot overlap. Chunks are swept in parallel. Gaps are then processed differently: PHast processes them serially, continuing for each gap the sweep of the previous chunk (whose state, including the priority queue, is preserved) after adding to its occupancy the positions of the first buckets of the following chunk; PHast-R sweeps all gaps in parallel, each with a new sweep in which the positions of the buckets of the neighboring chunks (on both sides) are marked as occupied and *not evictable*, as repair must not move buckets owned by another chunk.

Table 1: Comparison on 10^7 keys (single-threaded construction). d is the maximum depth of repair ($d = 0$: no repair).

Method	Configuration	bits/key	build [ns/key]	query [ns]
PHast ⁺	$S = 8, \lambda = 5.25$	2.116	40	22.8
PHast ⁺ wrap $\delta = 1$	$S = 8, \lambda = 5.25$	2.095	48	22.0
PHast ⁺ wrap $\delta = 2$	$S = 8, \lambda = 5$	2.011	59	20.9
PHast ⁺ wrap $\delta = 3$	$S = 8, \lambda = 5$	1.970	70	21.3
PHast	$S = 8, \lambda = 4.5$	1.923	562	19.9
PHast-R	$S = 8, L = 512, \lambda = 5, d = 0$	2.022	48	22.2
PHast-R	$S = 8, L = 512, \lambda = 5, d = 1$	1.935	79	21.4
PHast-R	$S = 8, L = 512, \lambda = 5, d = 2$	1.924	127	21.4
PHast ⁺	$S = 10, \lambda = 5.15$	2.085	38	19.7
PHast ⁺ wrap $\delta = 1$	$S = 10, \lambda = 6.2$	1.906	60	19.9
PHast ⁺ wrap $\delta = 2$	$S = 10, \lambda = 5.9$	1.884	87	19.7
PHast ⁺ wrap $\delta = 3$	$S = 10, \lambda = 6$	1.872	121	20.1
PHast	$S = 10, \lambda = 6.05$	1.854	1646	19.9
PHast-R	$S = 9, L = 1024, \lambda = 5.75, d = 1$	1.888	85	24.8
PHast-R	$S = 10, L = 2048, \lambda = 6, d = 0$	1.922	52	24.2
PHast-R	$S = 10, L = 2048, \lambda = 6, d = 1$	1.861	86	23.7
PHast-R	$S = 10, L = 2048, \lambda = 6.25, d = 2$	1.854	137	24.0
PHast-R	$S = 11, L = 4096, \lambda = 6.75, d = 1$	1.860	88	24.1

Summary of the differences with PHast⁺. Besides patterns and repair, PHast-R differs from PHast⁺ in the following respects: the seed encodes a pattern and a shift, and offsets are taken from a value o derived from the hash of the first level (Placement), also at the following levels, where offsets are given by o xored with the hash of the level (Levels); the slice length adapts to the size of the level, and it is longer for large levels (Slice length); the last level uses the same scheme as the other levels instead of regular PHast (Last level); and gaps between parallel chunks are swept in parallel by independent sweeps instead of serially by continuing the previous sweep (Parallel construction). Bucket priorities have the same form as in PHast⁺ (a weight depending on the size of the bucket, extrapolated linearly beyond size 7, minus 1024 times the index of the bucket), with retuned weights.

4 Experiments

We ran all experiments on an Apple M1 Max with 64 GiB of RAM, compiling with Rust 1.98 at optimization level 3 with thin link-time optimization. Keys are distinct 64-bit integers. The reference implementation is the `ph` crate by Beling (Function2 with the `ShiftOnly`, `ShiftOnlyWrapped< $\delta$ >` and `SeedOnly` seed choosers for PHast⁺, PHast⁺ with wrapping and PHast, respectively). To make query times comparable we fed both implementations the same hash function, XXH3-64 with a seed. Construction times are single-threaded unless otherwise noted and include hashing; query times are the best of three averages over 10^7 queries of random keys of the set. PHast-R uses $R = 4$ patterns, 16 repair candidates, and byte seeds for $S = 8$ and bit-packed seeds otherwise.

Table 1 and Figure 3 show the results on 10^7 keys. Patterns alone (PHast-R with $d = 0$) reduce space by 0.09 bits per key with respect to PHast⁺ at a cost of 7 ns per key. Repair is worth another 0.06–0.10 bits per key. With 8-bit seeds and depth 2, PHast-R uses the same space as PHast and builds 4.4 times faster; with 10-bit seeds and depth 2 it matches PHast in space and builds 12 times faster. Against the best PHast⁺ variant with wrapping, PHast-R with depth 1 uses 0.035 fewer bits per key with 8-bit seeds at 13% more construction time, and with

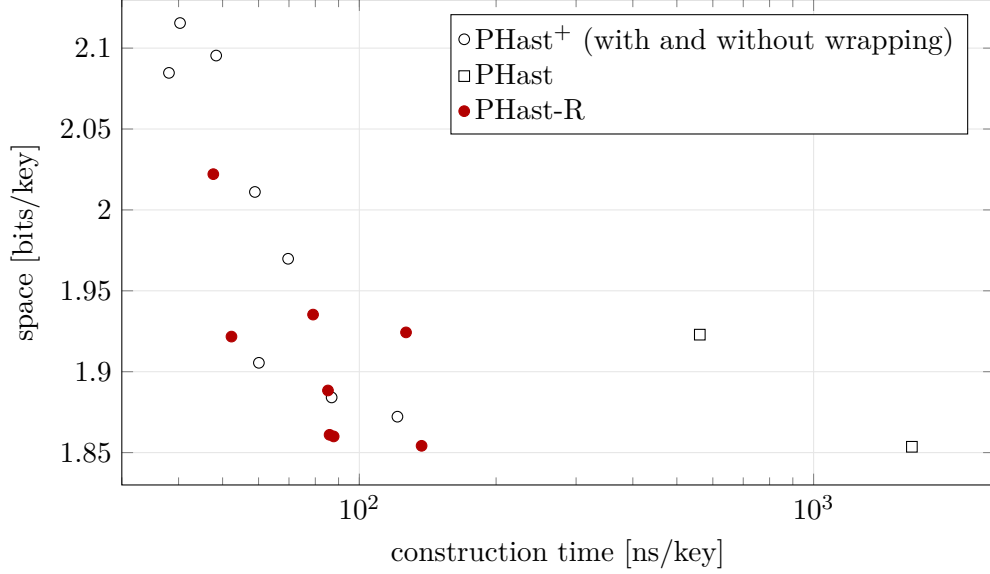


Figure 3: Space versus single-threaded construction time on 10^7 keys (data of Table 1).

Table 2: Comparison on 10^8 keys, with single-threaded and 10-threaded construction.

Method	Configuration	bits/key	build [ns/key]		query [ns]
			1 thread	10 threads	
PHast ⁺	$S = 8, \lambda = 5.25$	2.115	44	9.1	38.6
PHast ⁺ wrap $\delta = 3$	$S = 8, \lambda = 5$	1.968	75	13.6	34.9
PHast	$S = 8, \lambda = 4.5$	1.921	575	–	31.1
PHast-R	$S = 8, \lambda = 5, d = 1$	1.934	81	13.8	34.9
PHast-R	$S = 8, \lambda = 5, d = 2$	1.923	127	18.7	35.0
PHast ⁺ wrap $\delta = 3$	$S = 10, \lambda = 6$	1.869	126	19.2	32.5
PHast-R	$S = 10, \lambda = 6, d = 1$	1.857	87	13.0	37.3
PHast-R	$S = 10, \lambda = 6.25, d = 2$	1.850	137	19.4	38.3

10-bit seeds it is both smaller and 29% faster to build. Queries with byte seeds are as fast as in PHast⁺; bit-packed seeds currently cost about 4 ns more in our implementation.

Table 2 shows that the picture does not change on 10^8 keys, where queries are dominated by cache misses. With 10 threads (8 performance and 2 efficiency cores), PHast-R with depth 1 builds about as fast as PHast⁺ with wrapping and 8-bit seeds, and faster with 10-bit seeds: repair is local, so it parallelizes as well as the rest of the sweep.

5 The Limits of Repair

Since repair is responsible for a large part of the improvement of PHast-R over PHast⁺, it is natural to ask how far it can go: if we let repairs proceed with no bound on their depth or breadth, can we get rid of bumping altogether? In this section we show that the answer is negative, for two different reasons. The first reason is structural, and it applies to every placement scheme of the PHast family, with or without repair: no placement of the keys, however found, can avoid bumping. The second reason is algorithmic: most failures of the sweep cannot be repaired at any depth, and repairing the other ones makes the sweep fail more often.

5.1 A Lower Bound on Bumping

In all structures of the PHast family a key whose slice starts at σ can be placed only in the window $[\sigma \dots \sigma + W)$, where $W = L$ for PHast, $W = L + 2^S - 2$ for PHast⁺, and $W = L + D - 1$ for PHast-R. The seed of a bucket chooses positions within the windows of its keys, but the windows themselves depend only on the hashes of the keys. This simple constraint is sufficient to force bumping.

Proposition 1. *Consider n keys and n slots, and assume that each key x can be placed only in the slots $[\sigma_x \dots \sigma_x + W)$. Let*

$$X(t) = t - |\{x \mid \sigma_x < t\}| \quad (0 \leq t \leq n),$$

and let $R = \max_t X(t) - \min_t X(t)$ be the range of X . Then, every injective placement of a subset of the keys leaves at least $R - W + 1$ keys unplaced.

Proof. For $0 \leq s \leq t \leq n$ the number of keys x with $s \leq \sigma_x < t$ is $t - s - (X(t) - X(s))$. Only these keys can occupy the $t - s - W + 1$ slots in $[s + W - 1 \dots t)$, so at least $X(t) - X(s) - W + 1$ of these slots are empty. Symmetrically, these keys can occupy only the at most $t - s + W - 1$ slots in $[s \dots t + W - 1)$, so at least $X(s) - X(t) - W + 1$ of them are unplaced. Since there are as many keys as slots, the number of unplaced keys is equal to the number of empty slots, and it is thus at least $|X(t) - X(s)| - W + 1$ for all $s \leq t$. Choosing as s and t the points at which X attains its minimum and its maximum, in the appropriate order, completes the proof. $\square$

The proposition is a quantitative version of the (easy) necessity of Hall's condition [6] for the bipartite graph connecting keys and slots. In the PHast family slice starts are computed from a hash by inversion in fixed-point arithmetic, so we can model them as independent and uniformly distributed. If the σ_x are independent and uniform in $[0 \dots n)$, then $R = nV_n$, where V_n is Kuiper's statistic [8] of the values σ_x/n , that is, the sum of the maximum positive and negative deviations of their empirical distribution from the uniform distribution (the compression of slice starts into $[0 \dots n - W + 1)$ changes R by at most $W - 1$). By Donsker's theorem [2] and the continuity of the range, $R/\sqrt{n}$ converges in distribution to the range of a standard Brownian bridge, which by Vervaat's transformation [11] has the same distribution as the maximum of a standard Brownian excursion, whose mean is $\sqrt{\pi/2}$ [7]. Since the Dvoretzky–Kiefer–Wolfowitz inequality [10] gives $R/\sqrt{n}$ exponentially decreasing tails, the convergence extends to expectations, and we conclude that

$$\mathbf{E}[R] = \sqrt{\frac{\pi n}{2}}(1 + o(1)).$$

We verified the estimate experimentally: over 200 random sets of 2^{12} , 2^{16} , and 2^{20} keys, the average range is within 1% of $\sqrt{\pi n/2}$.

The bound is irrelevant in terms of space. For PHast-R with $S = 8$ and $L = 1024$ we have $W = 1087$, so on 10^7 keys at least about 2900 keys (0.03%) must be bumped, and on 10^9 keys about 38 500 (0.004%), whereas, as we will see shortly, the first level of PHast-R bumps about 2.6% of the keys. Its importance lies in the fact that it applies to *every* placement: as soon as $\sqrt{\pi n/2}$ is significantly larger than W , that is, beyond a few hundred thousand keys, no amount of search can make a first level with $m = n$ free of bumping. In particular, it rules out the possibility of obtaining a bump-free version of PHast-R, whose space would be just S/λ bits per key, by a backtracking search in the style of CONSENSUS [9], which combines the search and the encoding of seeds to get arbitrarily close to the lower bound for MPHFs. For such a search to succeed, keys should be first split into parts of exactly known size C , with $\sqrt{\pi C/2}$ significantly smaller than W , as it happens (albeit for different reasons) in Bucketed CONSENSUS-RecSplit, whose first step is a minimal k -perfect hash function.

Even within such parts, however, a second obstacle appears. Under the minimum-sum rule, whether a slot will be occupied is decided by buckets whose slices start up to W slots before it,

Table 3: Fraction of keys bumped from the first level of PHast-R, and space, as a function of the maximum depth d of repair (10^7 keys, $S = 8$, $L = 1024$, 16 candidates). These values do not depend on the hardware.

d	$\lambda = 4.75$		$\lambda = 5.25$	
	bumped	bits/key	bumped	bits/key
0	3.819%	2.0241	6.049%	2.0244
1	2.612%	1.9304	4.724%	1.9251
2	2.464%	1.9179	4.517%	1.9098
3	2.430%	1.9152	4.449%	1.9050
4	2.423%	1.9145	4.431%	1.9037

that is, about W/λ buckets before the slot becomes unreachable, so a depth-first search that detects a failure when a slot becomes unreachable must exhaust the alternatives of all such buckets before it can change the culprit. In a prototype using chained seeds as in CONSENSUS (the offsets of a bucket depend on the seeds of the previous buckets, so that backtracking rerandomizes the following ones), a cyclic output range to avoid boundary effects, and a check that all unreachable slots are occupied, the search never placed more than 160 buckets, even with more than two bits of slack per bucket and a budget of thousands of placements per bucket.

5.2 Repairable and Unrepairable Failures

The bump rate of PHast-R is two orders of magnitude larger than the lower bound of Proposition 1, so the latter is not what limits repair in practice. Table 3 shows the fraction of keys bumped from the first level of PHast-R as a function of the maximum depth of repair. The decrease is geometric: depth one removes more than 80% of the bumps that repair can remove at all, and each further level removes about three quarters of the remaining ones. The limit, about 2.42% for $\lambda = 4.75$ and 4.43% for $\lambda = 5.25$, is 63% and 73%, respectively, of the bump rate without repair. Breadth saturates in the same way: at depth one and $\lambda = 4.75$, the bump rate is 3.59%, 3.13%, 2.81%, 2.61%, 2.59%, and 2.59% with 1, 4, 8, 16, 32, and 64 candidates.

Thus, a failure is either *repairable*, in which case a short chain of evictions suffices, or *unrepairable*, in which case no chain of evictions exists within the region that repair can reach: the blockers of an unrepairable bucket are typically buckets whose windows lie mostly in the fully occupied region behind the frontier of the sweep, and thus cannot be moved.

Letting repairs proceed with no bounds, as in PtrHash [5], makes things worse. In a simulator of the sweep (with the placement of PHast-R on a cyclic output range) in which an evicted bucket evicts in turn the cheapest blockers (with a tabu list to avoid short cycles), chains of evictions drift backwards, as the owner of a blocking slot is usually a bucket whose slice starts earlier than that of the evicting one. When a chain reaches a bucket whose window is entirely occupied, every seed blocks several keys, every eviction causes several further evictions, and the process becomes supercritical: on 2^{16} keys with $\lambda = 5$, about 9% of the repairs did not terminate within 10^4 evictions, both with $m = n$ and with 5% of slack in the output range. Confining evictions to buckets near the frontier stops the drift, but repairs then either terminate after a few evictions or cycle forever.

Finally, repair has a feedback effect on the sweep. Since with $m = n$ holes and bumped keys are the same thing (Section 2), every repaired failure fills a slot that would otherwise have remained empty: the region around the frontier becomes denser, and more buckets fail at their first placement. The reduction of bumping is thus smaller than the fraction of failures that repair fixes: the sweep settles into a new equilibrium, which repair, as any increase in flexibility, moves, but without removing bumping.

In practice, then, depth one or two is not a compromise: from depth two to depth four the

bump rate decreases by less than 0.1 percentage points, and space by less than 0.007 bits per key, while construction time keeps increasing.

6 Conclusions

The excess space of PHast⁺ is the cost of bumping, and a large part of it is caused by the rigidity of additive placement. Independent offset patterns remove the rigidity without losing bit-parallelism, and a half adder over the same words that test feasibility turns failures into cheap, local cuckoo repairs. The resulting structure, PHast-R, has the query of PHast⁺, the space of PHast, and an intermediate construction time. Repair has, however, intrinsic limits: no placement in which keys move within windows of W slots can avoid bumping about $\sqrt{\pi n/2} - W$ keys, and in practice most failures of the sweep cannot be repaired at any depth, so shallow repairs capture almost all of the benefit.

There are several open problems and further research directions. On the practical side, the bucket priority should be retuned for larger seeds, access to bit-packed seeds should be made faster, and the last levels could be replaced by a more compact structure. More interestingly, the lower bound of Section 5 does not apply to keys split into parts of exactly known size: whether bump-free placements, which would use just S/λ bits per key, can be found efficiently within such parts, in spite of the long-range constraints that defeat backtracking, is an open problem.

The implementation is available in the `sux` crate as `sux::func::PHastR`.

References

- [1] Piotr Beling and Peter Sanders. PHast — perfect hashing made fast. *CoRR*, abs/2504.17918, 2025.
- [2] Patrick Billingsley. *Convergence of Probability Measures*. Wiley, second edition, 1999.
- [3] Peter Elias. Efficient storage and retrieval by content and address of static files. *Journal of the ACM*, 21(2):246–260, 1974.
- [4] Robert M. Fano. On the number of bits required to implement an associative memory. Memorandum 61, Computer Structures Group, Project MAC, MIT, Cambridge, Mass., n.d., 1971.
- [5] Ragnar Groot Koerkamp. PtrHash: Minimal perfect hashing at RAM throughput. In *23rd International Symposium on Experimental Algorithms (SEA 2025)*, volume 338 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 21:1–21:21. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2025.
- [6] P. Hall. On representatives of subsets. *Journal of the London Mathematical Society*, 10(1):26–30, 1935.
- [7] D. P. Kennedy. The distribution of the maximum Brownian excursion. *Journal of Applied Probability*, 13(2):371–376, 1976.
- [8] N. H. Kuiper. Tests concerning random points on a circle. *Proceedings of the Koninklijke Nederlandse Akademie van Wetenschappen, Series A*, 63:38–47, 1960.
- [9] Hans-Peter Lehmann, Peter Sanders, Stefan Walzer, and Jonatan Ziegler. Combined search and encoding for seeds, with an application to minimal perfect hashing. In *33rd Annual European Symposium on Algorithms (ESA 2025)*, volume 351 of *Leibniz International Proceedings in Informatics (LIPIcs)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2025. Article 109.

- [10] Pascal Massart. The tight constant in the Dvoretzky–Kiefer–Wolfowitz inequality. *The Annals of Probability*, 18(3):1269–1283, 1990.
- [11] Wim Vervaat. A relation between Brownian bridge and Brownian excursion. *The Annals of Probability*, 7(1):143–149, 1979.